

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 4
по курсу «Арифметические и логические основы вычислительной
техники»
на тему «Бинарное дерево поиска»

Выполнили
студенты группы 24ВВВ2:

Воеводин И.А.
Гуляевский Д.С.
Каташов К.А.

Приняли
Юрова О.В.
Деев М.В.

Пенза, 2025

Цель работы

Научиться работать с бинарными деревьями.

Лабораторное задание

Задание 1

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Задание 2*

1. Изменить функцию добавления элементов для исключения добавления одинаковых символов.

Описание метода решения

Метод решения основан на использовании бинарного дерева поиска (BST) для хранения и обработки чисел. Каждый элемент хранится в узле, который содержит значение и ссылки на левое и правое поддеревья.

Добавление элементов выполняется рекурсивно в методе `insert()`: число помещается в левое поддерево, если оно меньше текущего узла, и в правое — если больше. Поиск элемента реализован в методе `contains()`, который рекурсивно сравнивает заданное значение с ключами узлов.

Подсчёт числа вхождений (`num_of_app()`) выполняется обходом всего дерева и увеличением счётчика при совпадении значения. Метод `print()` выводит дерево в порядке возрастания.

В функции `main()` реализовано меню, позволяющее заполнять дерево, искать элементы, считать их количество, выводить содержимое и завершать программу.

Псевдокод

алг работа_с_бинарным_деревом

дано: пустое бинарное дерево поиска

надо: реализовать вставку, поиск, подсчёт вхождений и вывод элементов
нач

создать структуру узла (ключ, левый, правый)

инициализировать корень пустым

insert(node, key):

если node пуст → создать новый узел с key

иначе если $key < node.key$ → insert(node.left, key)

иначе → insert(node.right, key)

contains(node, key):

если node пуст → вернуть false

если $key = node.key$ → вернуть true

если $key < node.key$ → contains(node.left, key)

иначе → contains(node.right, key)

num_of_app(node, key, count):

если node пуст → вернуть

num_of_app(node.left, key, count)

если $node.key = key$ → count++

num_of_app(node.right, key, count)

print(node):

если node пуст → вернуть

print(node.left)

вывести node.key

print(node.right)

главная программа:

создать дерево bst

повторять:

вывести меню

если выбран пункт 1 → вводить числа до 0 и вставлять в дерево

если 2 → ввести число и вызвать поиск

если 3 → ввести число, подсчитать вхождения и вывести результат

если 4 → вывести дерево

если 5 → завершить

конец

Листинг

```
#include <iostream>
using namespace std;
class BST {
    struct Node
    {
        int key;
        Node* l = nullptr;
        Node* r = nullptr;
        Node(int key) : key(key) {};

        }*root = nullptr;

    bool contains(Node* n, int key) const {
        if (!n) {
            cout << key << " не было обнаружено" << endl;
            return false;
        }
        if (key == n->key) {
            cout << "найденно число " << key << endl;
            return true;
        }

        if (key < n->key)
            return contains(n->l, key);
        else
            return contains(n->r, key);

    }

    void num_of_app(Node* n, int key, int& count) const {
        if (!n) return;
```

```

        num_of_app(n->l, key, count);
        if (n->key == key)
            count++;
        num_of_app(n->r, key, count);
    }

```

```

void insert(Node*& n, int key) {
    if (!n) {
        n = new Node(key);
        return;
    }
    if (key == n->key)
        return;
    if (key < n->key)
        insert(n->l, key);
    else
        insert(n->r, key);
}

```

```

void print(Node* n) const {
    if (!n)
        return;
    print(n->l);
    cout << n->key << "\n";
    print(n->r);
}

```

public:

```

    bool contains(int key) const {
        return contains(root, key);
    }

```

```

}

void insert(int key) {
    insert(root, key);
}

void print() const {
    print(root);
}

void num_of_app(int key, int& count) const {
    count = 0;
    return num_of_app(root, key, count);
}
};

int main()
{
    setlocale(LC_ALL, "");
    BST bst;
    int choice, x, count;

    while (true) {
        cout << "\n===== МЕНЮ =====\n";
        cout << "1. Заполнить дерево числами\n";
        cout << "2. Найти элемент\n";
        cout << "3. Подсчитать число вхождений элемента\n";
        cout << "4. Показать дерево\n";
        cout << "5. Выход\n";
        cout << "Выберите пункт: ";
        cin >> choice;

        switch (choice) {

```

```

        case 1:
            cout << "Введите числа для вставки (0 — конец
ввода):\n";
            while (true) {
                cin >> x;
                if (x == 0)
                    break;
                bst.insert(x);
            }
            cout << "Дерево заполнено.\n";
            break;

        case 2:
            cout << "Введите число для поиска: ";
            cin >> x;
            if (bst.contains(x))
                cout << "Число " << x << " найдено в дереве.\n";
            else
                cout << "Число " << x << " не найдено.\n";
            break;

        case 3:
            cout << "Введите число для подсчёта вхождений: ";
            cin >> x;
            count = 0;
            bst.num_of_app(x, count);
            cout << "Число " << x << " встречается в дереве " <<
count << " раз(a).\n";
            break;

        case 4:
            bst.print();
            break;

```

```
case 5:
    cout << "Выход из программы.\n";
    return 0;

default:
    cout << "Неверный пункт меню, попробуйте снова.\n";
    break;
}
}
}
```

Результат работы программы

```
1. Заполнить дерево числами
2. Найти элемент
3. Подсчитать число вхождений элемента
4. Показать дерево
5. Выход
Выберите пункт: 2
Введите число для поиска: 4
найденно число 4
Число 4 найдено в дереве.
```

```
===== МЕНЮ =====
1. Заполнить дерево числами
2. Найти элемент
3. Подсчитать число вхождений элемента
4. Показать дерево
5. Выход
Выберите пункт: 3
Введите число для подсчёта вхождений: 3
Число 3 встречается в дереве 1 раз(а).
```

Вывод

В ходе выполнения лабораторной работы реализовали программу, использующую бинарное дерево поиска для хранения и обработки числовых данных. В процессе работы мы изучили принципы построения и обхода дерева, реализовали функции добавления, поиска, подсчёта количества вхождений и вывода элементов в упорядоченном виде.